Implementation Report

Introduction

This repository implements the paper *Neural Execution of Graph Algorithms* [1] (ICLR 2020), with the objective of reproducing its main experimental results. The paper studies whether neural networks can learn to execute classical graph algorithms by predicting the next algorithmic state at each iteration.

The implemented model is trained to imitate the step-by-step execution of several graph algorithms, including BFS, Bellman-Ford, and Prim's algorithm. In addition to reproducing the original tasks, this work extends the framework to the Connected Components algorithm.

This report provides a concise overview of the implementation, experimental setup, and results. It is intended to serve both as documentation for the repository and as supporting notes for understanding the model.

Method

Graph Generation

Following the experimental setup of the original paper, the training data is generated from seven families of undirected random graphs:

- Ladder graphs
- 2D grid graphs
- Trees (generated using Prüfer sequences)
- Erdős–Rényi graphs
- Barabási–Albert graphs
- 4-community graphs
- 4-caveman graphs

Model Architecture

The repository implements the maximisationbased Message Passing Neural Network (MPNN) [2], which was reported as the best-performing model in the original paper.

The architecture consists of:

- an **encoder** f_A and a **decoder** g_A, both specific to each algorithm;
- a shared **processor** P;
- a **predecessor network** R for tasks that require predecessor prediction, such as Bellman-Ford and Prim;
- a **termination network** T_A predicting when the algorithm should stop.

Additional details can be found in [3] and the computation pipeline can be summarised as:

$$
\begin{aligned}
& \xrightarrow{\text{Decoder}} y^t = g_A(h^t, z^t) \\
(x^t, h^{t-1}, G) \xrightarrow{\text{Encoder}} z^t = f_A(x^t, h^{t-1}) \xrightarrow{\text{Processor}} h^{(t+1)} = P(z^t, G) \quad & \xrightarrow{\text{Predecessor}} \pi^t = R(h^t, G) \\
& \xrightarrow{\text{Termination}} \tau^{(t)} = T_A(\bar{h}^t)
\end{aligned}
$$

Let N be the number of nodes and K the latent dimension. The variables have the following shapes:

$$x \in \mathbb{R}^N, \quad z \in \mathbb{R}^{N \times K}, \quad h \in \mathbb{R}^{N \times K}, \quad y \in \mathbb{R}^N, \quad \pi \in \mathbb{R}^{N \times N}, \quad \tau \in \mathbb{R}.$$

The loss function depends on the type of prediction required by each algorithm. Prim's algorithm is trained to predict the next node in the minimum spanning tree (MST).

- Mean Squared Error (MSE): Bellman-Ford distances, Connected Components
- Binary Cross-Entropy (BCE): BFS reachability and termination prediction
- Categorical Cross-Entropy (CE): Prim's next node selection and predecessor prediction

Experimental setup:

- Latent dimension $K = 32$
- Optimizer Adam
- Learning rate 5×10^{-4}

The implementation is written in PyTorch 2.10.

Training

The model is trained on 700 graphs in total, composed of 100 graphs from each of the seven graph families. Validation is performed on 140 graphs, with 20 graphs sampled from each family.

Early stopping is applied based on validation performance, with a patience of 50 epochs. A minimum number of epochs is enforced because the early stages of training are unstable, mainly due to the termination network, which tends to converge more slowly than the other prediction tasks.

Training takes a few hours on a CPU/GPU setup (Intel Core i7-12700K / RTX 4060 Ti).

Evaluation

Evaluation is performed on 20 unseen random graphs with varying numbers of nodes (20, 50 and 100 nodes). Increasing the number of nodes allows us to assess how well the model has learned the algorithms, i.e., whether it is able to generalise algorithmic reasoning.

Results

This section compares the results obtained by the repository with the results reported in the original paper.

BFS

Model MPNN-max	Reachability (mean step accuracy / last-step accuracy)		
	20 nodes	*50 nodes*	*100 nodes*
Paper	100.0% / 100.0%	100.0% / 100.0%	99.92% / 99.80%
Repository	100.0% / 100.0%	100.0% / 100.0%	100.0% / 98.13%

Bellman-Ford

Model MPNN-max	B-F (mean squared error / mean termination accuracy)		
	20 nodes	*50 nodes*	*100 nodes*
Paper	0.005 / 98.89%	0.013 / 98.58%	0.238 / 97.82%
Repository	0.013 / 93.33%	0.034 / 89.99%	0.317 / 86.57%

Model MPNN-max	B-F Predecessor (mean step accuracy / last-step accuracy)		
	20 nodes	*50 nodes*	*100 nodes*
Paper	97.13% / 96.84%	94.71% / 93.88%	90.91% / 88.79%
Repository	92.59% / 91.58%	88.48% / 81.22%	85.52% / 80.18%

Prim

Model MPNN-max	Prim Accuracy (next MST node / MST predecessor)		
	20 nodes	*50 nodes*	*100 nodes*
Paper	90.56% / 93.63%	52.23% / 88.97%	41.37% / 90.02%
Repository	94.74% / 75.26%	48.57% / 55.67%	27.75% / 89.90%

Extension

Details

The framework requires algorithms to be Markovian processes, i.e., the next state depends only on the current state. Algorithms such as DFS are not directly Markovian under a simple state representation and therefore cannot be trained in this framework without additional state information. Dijkstra's algorithm can be reformulated in a Markovian way by augmenting the node features with a visited state, but this would significantly increase the complexity of the implementation.

A natural extension of the framework is the Connected Components algorithm (CC), which can be formulated as a Markovian process as follows:

$$\text{CC:} \quad x_i^{(1)} = i \qquad x_i^{(t+1)} = \min\left(x_i^{(t)}, \min_{j \in \mathcal{N}_i} x_j^{(t)}\right)$$

Results

Connected Components is closely related to BFS, so strong results are expected. It can be interpreted as a multi-source BFS. Unlike the original paper, which primarily focuses on connected graphs, the algorithm has been tested on sparser graphs with multiple components to fully assess the performance of the algorithm.

Model MPNN-max	**CC (Component accuracy / Termination accuracy)**		
	20 nodes	*50 nodes*	*100 nodes*
Repository	99.67% / 98.94%	96.86% / 94.75%	95.32% / 92.40%

Comments

The termination prediction is the most challenging component of the model and gives the least satisfactory results (this component was dropped in the follow-up work [3]).

Overall, the model successfully captures simple algorithmic patterns (e.g. BFS), but performance degrades on more complex tasks (e.g. Bellman-Ford, Prim), especially as graph size increases. In particular, the predicted distances lack precision, which is partially masked by the use of an MSE loss, as it does not strongly penalise small but structurally incorrect deviations.

References

[1] Petar Veličković, Rex Ying, Matilde Padovano, Raia Hadsell, and Charles Blundell. Neural execution of graph algorithms. *arXiv*, 2019.

[2] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. *arXiv preprint arXiv:1704.01212*, 2017.

[3] Petar Veličković, Adrià Puigdomènech Badia, David Budden, Razvan Pascanu, Andrea Banino, Misha Dashevskiy, Raia Hadsell, and Charles Blundell. The clrs algorithmic reasoning benchmark. *arXiv*, 2022.